

RAD 2026 Implementeringsprojekt

Andy Kaarup Harboe, KQR508
Caroline Juliet Brinkmann Schwartz, QPZ655
Jora Ismaili, TCP541

1. juni 2026

Opgave 1c

For $n = 2^{24}$ og $l = 15$ fik vi, at den gennemsnitlige køretid over 5 forskellige streams med 5 forskellige instanser af parametrene a og a, b for Multiply-Shift og Multiply-Mod-Prime var hhv. 300,74 ms og 4477,16 ms. Altså så vi i gennemsnit en forskel på knap en faktor 15 mellem de 2 hashfunktioners køretider, hvor Multiply-Shift var klart hurtigst.

Vi har forsøgt at forklare denne forskel med udgangspunkt i typen af aritmetik og antallet af aritmetiske operationer. Med typen af aritmetik refererer vi her til, at Multiply-Shift bruger uint64 og Multiply-Mod-Prime bruger bigint. Dette er relevant, da uint64 bruger 64 bits og kan dermed være i én register, hvormed operationer er hurtige, mens bigint bruger et arbitrært antal bits, som derfor nemt kan være mere end 64, hvormed operationer vil kræve flere clock cycles at udføre. Antallet af operationer er åbenlyst relevant, for hvis alt andet var lige, ville flere operationer betyde mere arbejde og dermed højere køretid. Med en naiv og konservativ optælling af operationer, fås at Multiply-Mod-Prime bruger ca. 3-4 gange flere aritmetiske operationer end Multiply-Shift.

Opgave 3

Nedenfor er køretiden for Multiply-Shift og Multiply-Mod-Prime vist i sekunder, for forskellige $l = 10, \dots, 24$ med $n = 2^{24}$. Hver af køretiderne er et gennemsnit over 5 runs med 5 forskellige streams og 5 forskellige instanser af hashfunktions parametrene.

Som beskrevet i opgavebeskrivelsen ser vi ud fra resultaterne, at desto større l bliver, desto mindre bliver betydningen af valget af hashfunktion på køretiden. Sagt med andre ord ser vi, at når l stiger, falder forskellen i køretider mellem Multiply-Shift og Multiply-Mod-Prime relativt til den samlede køretid. Eksempelvis ser vi for $l = 10$, at $(5,151 - 0,849)/(5,151 + 0,849) = 0,717$, mens for $l = 24$ ses, at $(45,694 - 24,475)/(45,694 + 24,475) = 0,302$.

l	Multiply-Shift (s)	Multiply-Mod-Prime (s)
10	0,849	5,151
11	0,833	5,325
12	0,866	5,500
13	0,939	5,672
14	1,101	6,038
15	1,340	7,256
16	1,883	8,331
17	2,846	10,167
18	3,947	11,208
19	5,793	13,083
20	6,972	13,325
21	8,141	15,360
22	9,118	19,283
23	14,160	26,888
24	24,475	45,694

Vi har forsøgt at forklare faldet i den relative forskel i køretid ved at betragte CPU'ens cache. Når l øges, stiger størrelsen af hashtabellen (antal buckets) eksponentielt med 2^l , hvilket betyder, at CPU'ens cache vil blive fyldt op, og køretiden vil blive domineret af cache misses og den tilhørende fetch fra L3 cache eller RAM, som er betydeligt langsommere. Ligeledes når l øges, stiger antallet af forskellige nøgler også eksponentielt med 2^l , hvormed vi sandsynligvis vil opleve, at når vi gennemløber streamen, vil to efterfølgende nøgler have hver deres bucket langt fra hinanden, hvilket ligeledes fører til flere cache misses og flere fetches fra L3 cache eller RAM. Altså vil køretiden for større l domineres af cache misses og dertilhørende fetches fra memory længere nede i memory-hierarkiet.

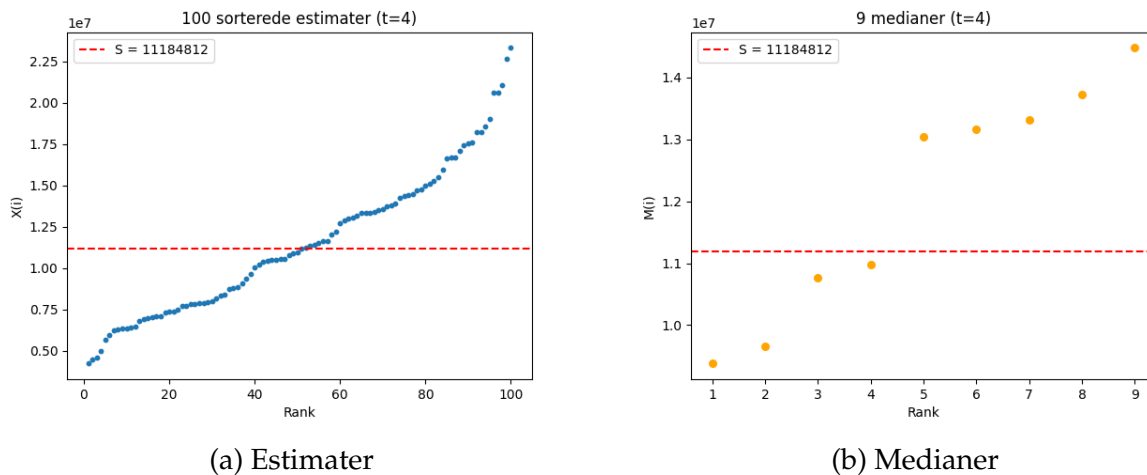
Vi har valgt, at det første l , hvor køretiden blev for stor, er $l = 24$. Dette skyldes, at vi kan ikke teste større l uden også at øge n til mindst 2^l , hvilket betyder at køretiden af eksperimenterne med mindre l 'er vil stige markant grundet, at n er steget, hvormed der vil gælde, at det samlede eksperiment vil tage for lang tid at køre. Ligeledes har vi valgt den maksimale l , så den samlede køretid for de næste opgaver ikke bliver alt for stor.

Opgave 7

Resultaterne af eksperimenterne $X_{(1)}, \dots, X_{(100)}$ er præsenteret på figur 1a, hvor S også er indsat, for $n = 2^{24}$ og $l = 23$, da vi valgte maksimal l til 24 i forrige opgave.

Forventningsværdien af X er per opgavebeskrivelsen givet til $\mathbb{E}[X] = S \approx 11,184 \times 10^6$. Betragter vi den empiriske forventningsværdi $\hat{\mu} = \frac{1}{100} \sum_{i=1}^{100} X_i \approx 11,542 \times 10^6$, ser vi, at der er en forskel mellem den faktiske og empiriske forventningsværdi på cirka $0,36 \times 10^6$.

Variansen af X er per opgavebeskrivelsen givet til $\text{Var}(X) \approx 2S^2/m \approx 2(11,184 \times 10^6)^2/2^4 = 15,637 \times 10^{12}$. Betragter vi nu den empiriske varians, altså *mean squared error*



Figur 1: Opgave 7

(MSE), fås $\hat{\sigma}^2 \approx 19,184 \times 10^{12}$. Så vi ser en forskel mellem den teoretiske og empiriske variansværdi på cirka $3,547 \times 10^{12}$.

På figur 1b kan vi se, hvis vi sammenligner med figur 1a, at medianerne generelt ligger meget tættere på S , da de svinger mellem $0,9 \times 10^7$ og $1,5 \times 10^7$, hvorimod $X_{(1)}, \dots, X_{(100)}$ svinger mellem $0,4 \times 10^7$ og $2,3 \times 10^7$. Denne forbedring i estimatet forklarer vi med median tricket, som signifikant forbedrer værdien af vores estimater. Vi ser dog stadig en vis afvigelse - som sandsynligvis skyldes, at $m = 2^4$, hvilket er ret småt, og dermed resulterer i en vis usikkerhed - samt et pludseligt spring fra $M_{(4)}$ til $M_{(5)}$, hvilket nok blot skyldes en statistisk abnormitet.

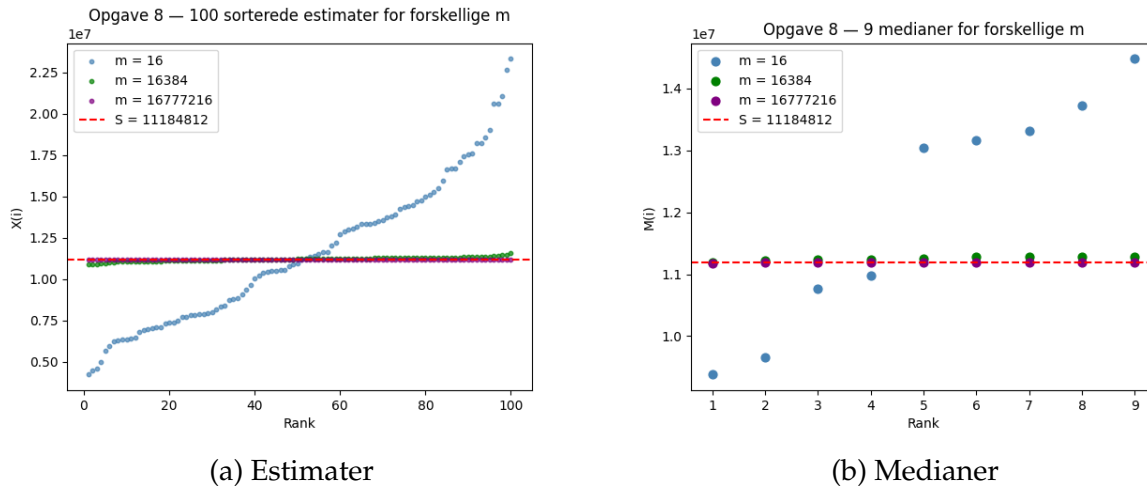
Opgave 8

På Figur 2a og 2b ses igen 100 (sorterede) estimater for S og medianer for (usorterede) grupperinger af disse, hvor Count-Sketch er kørt på samme datastrøm som ovenfor, men nu med tre forskellige værdier af $m = 2^t$, hvor $t = \{4, 14, 24\}$.

Her er det tydeligt, at allerede for $m = 16.384 = 2^{14}$ er estimaterne markant tættere på den sande S end med $m = 16 = 2^4$, og med $t = 24$ og dermed $m = n = 16.777.216 = 2^{24}$ kan hverken medianerne eller de enkelte estimater adskilles fra den sande S (med det valg af y-akse der skal til for at kunne se alle estimaterne for $m = 16$). Dette viser klart, at større m giver bedre præcision.

Ligeledes kan vi se på resultaterne i Tabel 1, at både MSE og den teoretiske varians $\text{Var}(X) \approx 2S^2/m$ falder med en faktor 10^3 når m stiger med samme faktor, og at MSE og den teoretiske varians stemmer mere og mere overens som m stiger.

I Tabel 1 er den gennemsnitlige køretid for Count-Sketch på tværs af de 100 eksperimenter også rapporteret. Her ser vi, at den ikke ændres betydeligt fra $m = 16$ til $m = 16.384$, og først når m øges til at være lig n stiger køretiden fra $\sim 8,4$ sekunder til $\sim 10,1$ sekund.



Figur 2: Opgave 8

Alle tre gennemsnitskøretider er dog stadig signifikant lavere, end når vi bruger hashing med chaining. Her tager det (med $n = 2^{24}$ og $l = 23$) 14,16 sekunder ved brug af Multiply-Shift og 26,89 sekunder ved brug af Multiply-Mod-Prime.

Vi kan dermed se, at uanset valget af m (i det range, som vi har testet) får vi en markant lavere køretid ved at bruge Count-Sketch end med hashing med chaining. Fokuserer vi dermed på tilfældet hvor $m = n$ ser vi ydermere, at approksimationen af S bliver meget god. Vi har en sand $S = 11.184.812$, og med $m = n$ er min. og max. af de 100 estimer henholdsvis 11.176.092 og 11.194.348. Når vi kigger på de 9 medianer i Figur 2b (over 11 runs hver) afviger de med maksimalt 2536, altså blot 0,02%, fra den sande S .

m	MSE	$2S^2/m$	Køretid (s)
16	$19,184 \times 10^{12}$	$15,637 \times 10^{12}$	8,454
16.384	$15,253 \times 10^9$	$15,270 \times 10^9$	8,411
16.777.216	$14,927 \times 10^6$	$14,913 \times 10^6$	10,138

Tabel 1: Count-Sketch stats for forskellige m med $n = 2^{24}$, $l = 23$, $S = 11.184.812$

Kode

Selve implementeringen og den producerede data kan findes på dette Github [repo](#).